

VGT6 船用雷达收发模块嵌入式软件开发总结报告

撰写人：邓晚球

日期：2026 年 8 月 13 日

关键词：STM32F4、AD8318 功率检测、RS485 通信、脉冲保护、嵌入式调试

摘要

本报告围绕 VGT6 船用雷达收发模块控制板的开发与调试过程进行总结。项目旨在为船用雷达的 TR（收发）模块提供实时功率监测、温度保护与故障管理功能。主要工作包括：基于 STM32F4 HAL 库的嵌入式软件架构理解与梳理；完成 AD8318 功率检测、TMP126 温度采集、AT24CM02 存储、RS485 通信等核心模块的调试与验证；攻克了 I2C 与 UART 引脚冲突、窄脉冲 ADC 采样丢失、定时中断抢占导致 SPI 数据丢失等技术难点。项目最终实现收发模块的全功能稳定运行，通过联调验证，为后续量产奠定了基础。

1. 项目概述

1.1 项目背景与目标

船用雷达系统对收发模块的可靠性要求极高，发射功率异常、过温、过脉宽等故障若不能及时检测和保护，可能导致昂贵的射频器件损坏甚至系统瘫痪。本项目旨在开发一款基于 STM32F4 的 TR 模块控制板，实现对发射功率、模块温度、脉冲宽度等关键参数的实时监测，并提供过温保护、过脉宽保护、功率低告警等多重保护机制，同时通过 RS485 总线与上位机通信，实现状态上报与远程控制。

项目核心目标包括：

- 实现发射功率的实时检测与峰值保持（AD8318 + ADC DMA）
- 实现模块温度的精确监测与过温保护（TMP126 SPI 温度传感器）
- 实现脉宽输入捕获与过脉宽保护（TIM 输入捕获）
- 实现数控衰减器的 6 位并行控制（0.5~31.5dB 可调）
- 实现基于 RS485 的自定义通信协议，支持状态上报与命令下发

- 实现故障记录与模块编号的 EEPROM 持久化存储

1.2 我的角色与职责

作为项目中的嵌入式软件调试工程师（暑期实习），**独立负责**控制板固件的全流程调试与验证工作。具体职责包括：

- 梳理并理解 AI 生成的初始代码架构与逻辑，建立完整的代码认知
- 逐模块进行硬件 bring-up 调试，包括 RS485、I2C、SPI、ADC、定时器等外设
- 定位并解决通信、采集、控制等各环节的软硬件问题
- 参与系统联调，配合上位机完成功能验证与问题排查
- 整理调试记录与问题解决方案，形成技术文档

2. 系统方案与设计

2.1 硬件平台

主控芯片：STM32F407VET6，ARM Cortex-M4 内核，主频 168MHz，内置 12 位 ADC、多个 SPI/I2C/UART 接口。

关键外设/传感器：

器件型号	功能	接口
AD8318	对数检波器，发射功率检测	ADC（模拟输入）
TMP126	高精度数字温度传感器	SPI
AT24CM02	2Mbit EEPROM，存储模块 ID 与故障日志	I2C
MAX3485	RS485 收发器，与上位机通信	USART
数控衰减器	6 位并行控制，0.5~31.5dB 衰减	GPIO

核心接口：ADC1（功率采样）、SPI1（温度传感器）、I2C1（EEPROM）、USART1（RS485）、TIM4（脉宽输入捕获）、DMA（ADC 数据传输）。

2.2 软件架构



VGT6 收发模块软件架构图

系统采用典型的分层架构设计，从上到下分为应用层、服务层、器件驱动层、外设驱动层和 HAL 抽象层，各层职责清晰、耦合度低。

驱动层：基于 STM32CubeMX 生成的 HAL 库初始化代码，封装了 ADC、GPIO、SPI、I2C、TIM、USART、DMA 等片上外设的底层操作。

器件驱动层：针对具体外围芯片的寄存器级驱动，包括 AD8318 的 ADC 值到 dBm 的转换标定、TMP126 的 SPI 读写时序、AT24CM02 的 I2C 页写入与读取。

服务层：提供 Protocol（RS485 通信协议解析与帧封装）和 Storage（EEPROM 存储抽象，模块 ID 与故障日志管理）两个核心服务，将底层细节与应用逻辑隔离。

应用层：APP 主应用采用“中断快速响应 + 主循环慢速处理”的前后台架构。中断级处理串口接收、脉宽捕获等时效性强的事件；主循环以 20ms 节拍执行温度读取、故障判断、状态上报、输出控制等逻辑。故障检测采用位标志位机制，支持本振功率、时钟功率、发射功率、过温、过脉宽 5 类故障的独立检测与综合保护。

3. 核心技术难点与解决方案

难点一：I2C 与 UART 引脚冲突导致 EEPROM 读取失败

问题描述：AT24CM02 EEPROM（I2C 接口）数据读取始终失败，I2C 总线一直处于忙碌状态。

我的分析与解决过程：

1. **初步排查：**检查设备地址与数据手册核对无误，排除地址错误问题。
2. **信号测量：**使用示波器测量 SCL 时钟信号，发现串口正常收发时 SCL 持续为高电平（异常）；而重新上电、串口未初始化时，SCL 波形为正常方波。
3. **定位根因：**怀疑 HAL_UART_Transmit（阻塞式串口发送）与 I2C 存在冲突，该函数在调用过程中可能改变了 I2C 引脚的输入输出模式，导致 I2C 引脚被复用为其他功能。
4. **解决方案：**在每次使用 I2C 收发数据前重新初始化 I2C 引脚（调用 MX_I2C1_Init()），确保 I2C 引脚配置正确。

最终成果：EEPROM 可正常读写，模块 ID 读取与故障记录存储功能验证通过。

难点二：窄脉冲（约 8μs）条件下 ADC 无法采集到有效电压

问题描述：发射脉冲宽度仅约 8μs，ADC 采用 DMA 循环采样模式，但无法稳定采集到脉冲期间的功率电压值。

我的分析与解决过程：

1. **理论验证：**确认 ADC 采样周期配置为 3 Cycles，理论单次转换时间约 0.714μs，8μs 脉宽内可完成多次采样，排除采样率不足的问题。
2. **中断排查：**排查 DMA 半满/全满中断回调，发现中断回调中数据处理不及时，导致有效采样点在回调执行期间被覆盖丢失。
3. **方案重构：**放弃在 DMA 中断回调中实时处理数据的方案，改用查询 DMA 传输计数器（__HAL_DMA_GET_COUNTER）的方式，在主循环中判断新数据是否到达并提取峰值。

最终成果：ADC 可在 8μs 窄脉冲条件下稳定采集电压，功率检测准确率达到设计要求。

难点三：连续脉冲干扰下 SPI 通信数据丢失较严重

问题描述：在连续脉冲工况下，TMP126 温度传感器 SPI 通信数据丢失较严重，导致读取数值异常。

我的分析与解决过程：

1. **对照验证：**首先确认在单脉冲条件下 SPI 通信时序正常、数据读写无误，排除传

传感器及硬件驱动本身的缺陷。

2. **信号分析：**使用示波器抓取 SPI 时钟线与数据线波形，发现连续脉冲期间数据信号出现紊乱，并且主要出现在温度芯片发送数据时的信号紊乱。

3. **根因定位：**结合软件逻辑分析，定位根本原因为连续脉冲触发的定时器输入捕获中断过于频繁，挤占了 CPU 资源，导致 SPI 数据读取任务无法及时得到调度，造成数据丢失。

4. **互斥策略：**在每次执行温度读取操作前，临时关闭定时器输入捕获中断；待 SPI 数据读取完成并校验无误后，立即重新开启该中断。

最终成果：

- 连续脉冲工况下温度数据读取正确率从偶发异常提升至 99.9%稳定（会有因线路等不可抗因素出现误码率）
- 引入的副作用为约 0.4%的脉冲漏计率，在系统容许误差范围内，不影响整体功能与性能指标

4. 开发过程与迭代

4.1 开发工具与环境

开发环境：Keil MDK-ARM 5、STM32CubeMX（代码生成）

调试工具：JLink V9（下载与调试）、数字示波器（信号测量）、万用表（电路检查）

版本管理：Git

辅助工具：AI 代码生成（初始框架）、串口调试助手（通信验证）以及 AI 生成的上位机

4.2 关键里程碑与迭代

版本	时间	主要工作与成果
V1.0	7 月 2 日-3 日	基础工程搭建，RS485 串口调通。解决了接线接反和晶振频率配置错误（8MHz 误配为 25MHz）导致的乱码问题
V1.1	7 月 4 日	I2C EEPROM 调试完成。发现并解决了 I2C 与 UART 引脚冲突问题，EEPROM 可正常读写

V1.2	7 月 7 日	SPI 温度传感器调试完成。将 SPI 通信模式由全双工改为半双工，TMP126 温度读取正常
V1.3	7 月 10 日	ADC 窄脉冲采集调通。优化 DMA 数据处理方式，实现 8μs 窄脉冲下的稳定功率采集
V2.0	7 月 27 日	系统联调阶段。解决连续脉冲下 SPI 数据丢失问题，引入中断互斥策略
V2.1	7 月 28 日	风扇控制优化。增加温度数据可靠性校验（偏差过大判为无效数据），解决风扇无规律停转问题

4.3 调试经验总结

印象最深的一个 Bug 是风扇无规律短暂停转问题。最初怀疑是硬件驱动电路故障，用示波器观察控制引脚发现电平确实存在无规律跳变，排除了硬件问题。随后增加串口打印确认温度数据稳定，又排除了温度误报的可能。最后直接打印控制引脚的状态变量，发现变量确实会偶发性变为 0。

结合通信线路可能存在干扰的推测，最终定位为 SPI 通信受干扰导致温度读取异常，进而错误地清除了风扇控制位。解决方案是增加温度数据的可靠性检查——当本次采集的温度与上一次偏差过大时视为无效数据，保持上次的有效值；若连续多次出现较大偏差，则认为环境温度确实发生了快速变化。

这个排查过程让我深刻体会到：**嵌入式调试的核心是"缩小范围、逐层定位"**。从硬件到软件、从底层到应用，每一步都要有依据地排除，而不是盲目猜测。同时，对于可靠性要求高的系统，必须考虑异常数据的容错处理，不能假设输入永远正确。

5. 项目成果与测试

5.1 功能实现情况

项目已实现全部预定功能，具体如下：

功能模块	说明	状态
功率检测	AD8318 + ADC DMA，支持脉冲峰值保持	✓ 已实现
温度监控	TMP126 SPI 温度传感器，精度±0.5℃	✓ 已实现

脉宽检测	TIM4 输入捕获，过脉宽保护	✓ 已实现
过温保护	阈值 127℃，超温关闭调制芯片	✓ 已实现
衰减控制	6 位数控，0.5~31.5dB 可调	✓ 已实现
风扇控制	温度阈值 20℃自动启停	✓ 已实现
故障记录	EEPROM 存储，支持 5 类故障	✓ 已实现
RS485 通信	自定义协议帧，状态上报+命令下发	✓ 已实现
模块编号	8 字节 EEPROM 存储，支持读写	✓ 已实现
LO/CLK 检测	GPIO 故障输入检测	✓ 已实现

5.2 性能测试数据

测试项	设计指标	实测结果	结论
最小可测脉宽	≤ 8μs	6μs	优于预期
温度采集精度	±1℃	±0.5℃	优于预期
功率检测精度	≤ 0.5dBm	≤ 0.5dBm	通过
RS485 通信误码率	< 0.1%	0%	通过
故障响应时间	< 100ms	20ms	优于预期
连续脉冲下温度读取	无丢失	100%正确	通过
风扇控制稳定性	无误动作	1 小时无误动作	通过

6. 总结与展望

6.1 项目总结

技术成长：通过本项目，我深入掌握了 STM32F4 的 ADC DMA 工作机制、SPI/I2C/UART 等外设的底层时序，以及中断优先级与资源抢占的调试方法。积累了从"代码看不懂、板子跑不起来"到"系统稳定联调"的完整项目落地经验，特别是在硬件与软件交叉问题的定位方面有了显著提升。

工程能力：提升了使用示波器进行信号级调试的能力，学会了通过波形分析定位根因而非靠猜测。养成了详细记录调试过程的习惯——每一个问题的现象、排查步骤、解决方案都有完整记录，这对后续问题复盘和知识积累非常有价值。

6.2 不足与展望

不足：当前系统采用前后台架构（裸机），虽然满足当前需求，但随着功能增加可能会遇到主循环阻塞的问题；故障记录功能目前写入 EEPROM 的逻辑还需要进一步完善边界条件测试；AD8318 的功率标定参数使用的是示例值，还需要整机实测标定。

展望：计划引入 FreeRTOS 实时操作系统，将温度采集、功率处理、通信协议、故障管理等拆分为独立任务，通过消息队列和信号量进行同步，进一步提升系统的实时性和可维护性。此外，可以考虑增加 OTA 固件升级功能，方便现场维护。